# Integrating a Gaussian in Fortran, C, Python and Julia

Same algorithm, four languages: how fast, how complex?

## 1. The problem and the analytic answer

$$\int_0^1 (1/\sqrt{2\pi})\, e^{-x^2/2}\, dx = \tfrac{1}{2}\, \mathrm{erf}(1/\sqrt{2}) = 0.34134474606854 3$$

$= \Phi(1) - \tfrac{1}{2}$, the familiar “34.13 %” of the 68–95–99.7 rule.

Numerically: composite **Simpson's rule**, $N = 10^8$ intervals, identical loop in every language.

# 2. The same loop, four ways

```c
// C
for (long i=1; i<n; i++)
    s += f(a+i*h) * ((i&1) ? 4.0 : 2.0);
```

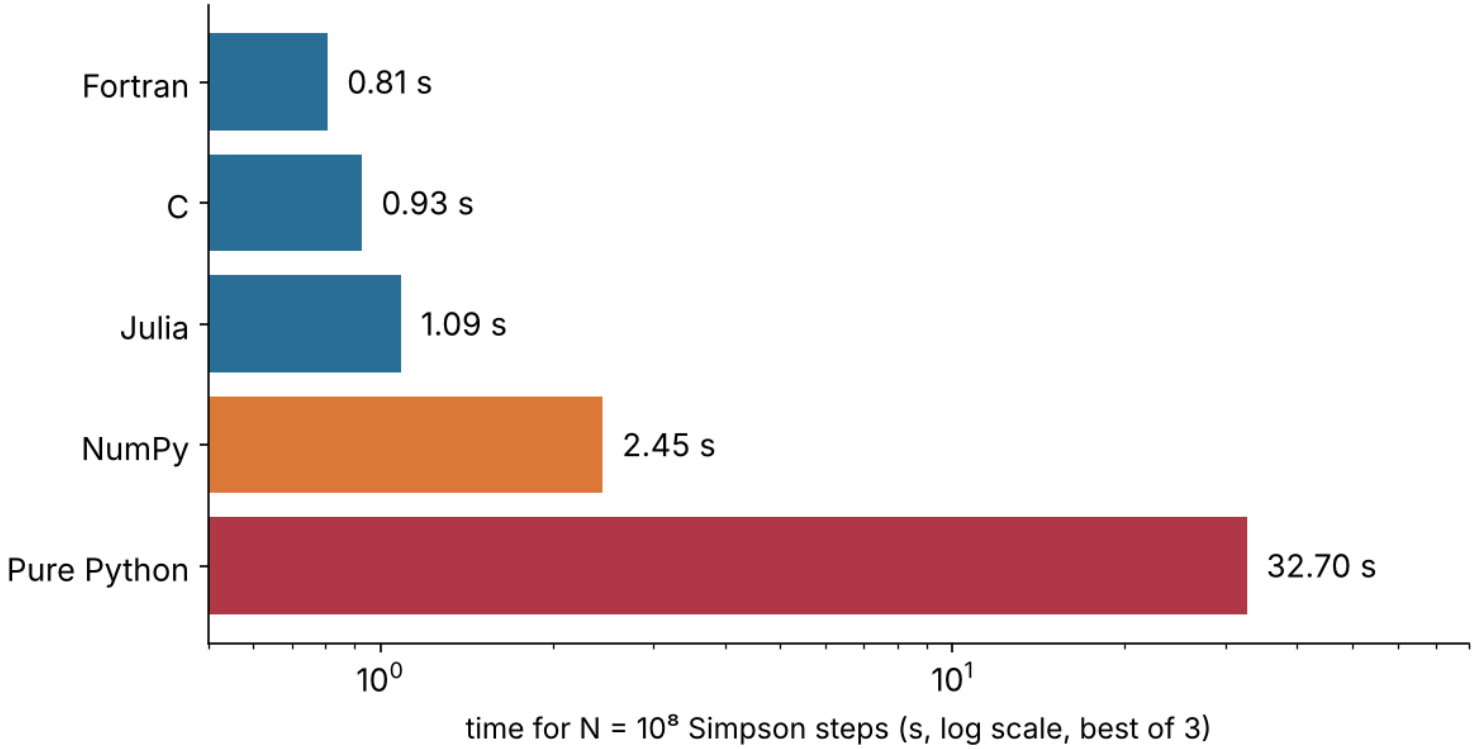
```fortran
! Fortran
do k = 1, n-1
   s = s + merge(4d0,2d0,mod(k,2_8)==1)*f(a+k*h)
end do
```

```python
# Python
for i in range(1, n):
    s += f(a+i*h) * (4.0 if i & 1 else 2.0)
```

```julia
# Julia
for i in 1:n-1
    s += f(a+i*h) * (isodd(i) ? 4.0 : 2.0)
end
```

Python/Julia: ~8 lines, read like the maths. C/Fortran: ~12–15 lines, explicit types and a compile step.

# 3. Speed (N = $10^8$, best of 3)



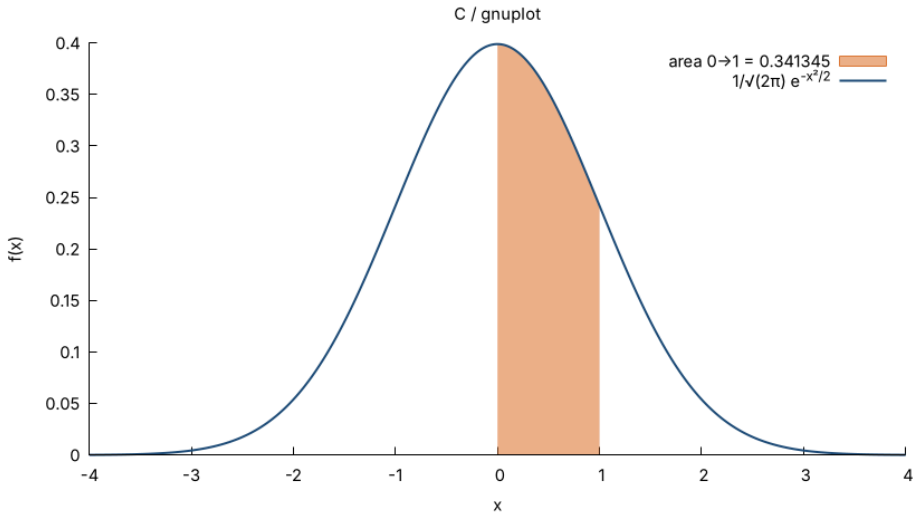

**C ≈ Fortran ≈ Julia** (~1 s): within noise, dominated by `exp()`.

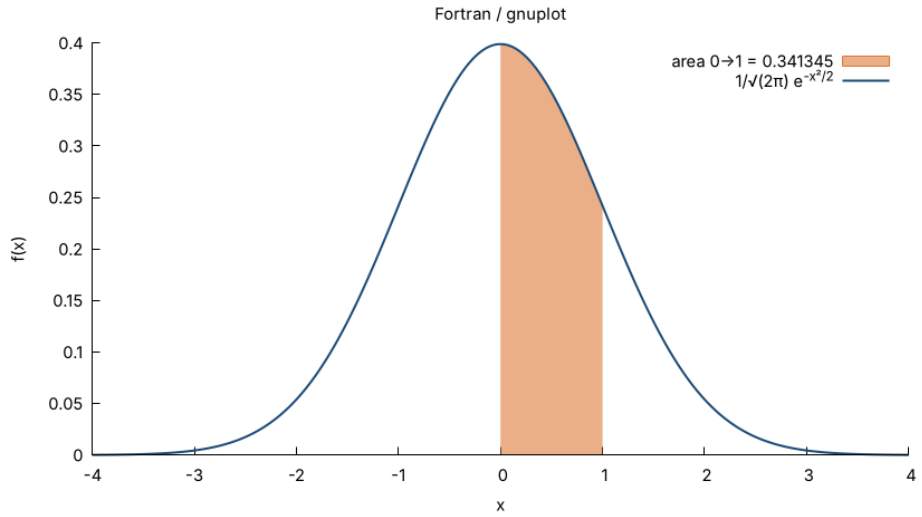
NumPy: ~2.5× slower ($10^8$-element temporaries).

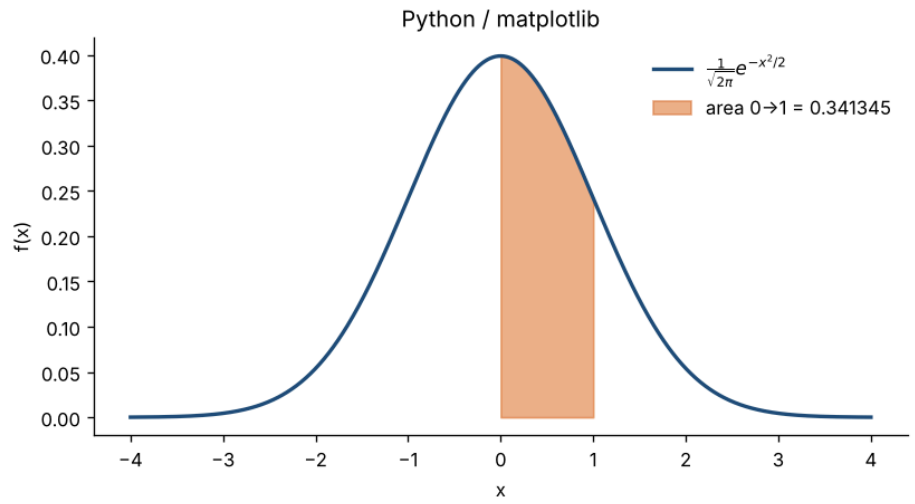
Pure Python: **~35× slower**.

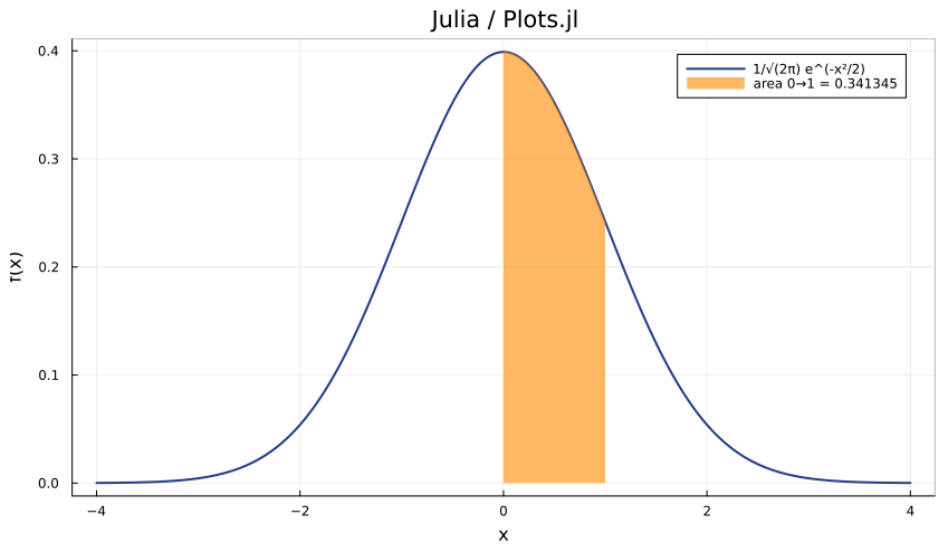
Shared 4-core cloud VM, timings noisy. Julia excludes JIT warm-up. Loop results 0.341344746068522: error $2 \cdot 10^{-14}$ is naive-summation round-off; NumPy's pairwise sum is exact to all digits.

# 4. Plot the area: four languages



# 5. Plotting: code and time

| Language | Tool | Wall time |
|---|---|---|
| C | gnuplot via popen | 0.06 s |
| Fortran | gnuplot via data file | 0.06 s |
| Python | matplotlib | ~1 s |
| Julia | Plots.jl (GR) | 3–4 s (startup + package load) |

```
ax.plot(x, g(x));  ax.fill_between(xs, g(xs), alpha=.6)        # Python
plot(x, g.(x));    plot!(xs, g.(xs), fillrange=0, fillalpha=.6)  # Julia
```

C and Fortran have no native plotting: they hand the data to gnuplot.

# Take-aways

- Compiled Fortran, C and Julia are equally fast; Julia matches them with Python-like code.
- NumPy recovers most of Python's speed; pure Python loops cost ~35×.
- Plotting: a different story. Startup cost matters more than compute for small jobs.
- Code, data and Makefile: **github.com/<you>/gauss-integral-benchmark**